

Duplicate Feature Maintenance Report

Student: Oskar / Tokushiro

Course: Software Maintenance

Project: <https://github.com/Tokushiro/JHotDraw>

Upstream case study: <https://github.com/sweat-tek/JHotDraw>

Branch: maintenance/duplicate-feature

Draft pull request: <https://github.com/Tokushiro/JHotDraw/pull/2>

CI push run: <https://github.com/Tokushiro/JHotDraw/actions/runs/27103448409>

CI pull-request run: <https://github.com/Tokushiro/JHotDraw/actions/runs/27103453227>

Date: 2026-06-07

Backlog evidence item: <https://github.com/Tokushiro/JHotDraw/issues/1>

1. Change Request

Requested Feature

Duplicate selected drawing elements.

User Story

As a drawing user, I want to duplicate selected drawing elements so that I can quickly reuse existing shapes without recreating them manually.

Details and Acceptance Expectations

- It must be possible to trigger the feature through the existing Duplicate edit action.
- If the active component is an enabled editable drawing component, the action must delegate to the component's `duplicate()` operation.
- Duplicated figures must preserve the source figure structure, be offset from the original selection, and become selected after the operation.
- The feature must not modify drawing state when no editable target is active.
- Existing Copy and Clear Selection action behavior must remain unchanged after refactoring shared action logic.

Repository and Environment Setup

The course lab repository was forked from `sweat-tek/JHotDraw` to `Tokushiro/JHotDraw` and cloned into:

C:\Users\oki13\Desktop\University\System Maintenance\Working on\JHotDraw-Duplicate-Maintenance

The work was performed on branch:

`maintenance/duplicate-feature`

The lab requested JDK 11 and Maven 3.8.x. The machine had Java 25 on PATH and no Maven on PATH, so portable tools were installed under the workspace:

- JDK: Temurin 11.0.31
- Maven: Apache Maven 3.8.8

Baseline build command:

```
mvn -s .maven-settings.xml clean install -DskipTests
```

Baseline result:

```
BUILD SUCCESS
```

GUI smoke command from `jhotdraw-samples/jhotdraw-samples-misc`:

```
mvn -s ../../.maven-settings.xml exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"
```

GUI smoke result:

- The process was still running after 12 seconds, which indicates that the Swing application had started and was waiting for user interaction.
- Console output was written to `report/gui-launch.log`.
- The output contained only resource icon warnings, for example missing optional `edit.duplicate.icon`, and no startup exception.

2. Concept Location

The Duplicate feature was located by searching for the `edit.duplicate` action id and calls to `duplicate()`.

Main feature entry point:

`jhotdraw-actions/src/main/java/org/jhotdraw/action/edit/DuplicateAction.java`

The action resolves the active Swing component and, when it is an enabled `EditableComponent`, delegates to:

```
((EditableComponent) c).duplicate();
```

The feature contract is defined by:

`jhotdraw-api/src/main/java/org/jhotdraw/api/gui/EditableComponent.java`

The drawing-view implementations that perform the real duplication are:

- `jhotdraw-core/src/main/java/org/jhotdraw/draw/DefaultDrawingView.java`
- `jhotdraw-core/src/main/java/org/jhotdraw/draw/AbstractDrawingView.java`

The implementation follows these steps:

1. Sort the current selected figures through the drawing.
2. Clear the current selection.
3. Clone each selected figure.
4. Translate each clone by (5, 5).
5. Add the clones to the drawing.
6. Remap cloned figure relationships using the original-to-duplicate map.
7. Select the duplicated figures.
8. Register an undoable edit that removes duplicates on undo and adds them on redo.

Initial Domain Class Responsibility Table

Domain class	Responsibility in Duplicate feature
<code>`DuplicateAction`</code>	User-facing edit action that starts the Duplicate feature from menus/toolbars/focus context.
<code>`AbstractSelectionAction`</code>	Shared base class for edit actions that operate on a selected or focused component.
<code>`EditableComponent`</code>	Contract exposing <code>`duplicate()`</code> , <code>`delete()`</code> , <code>`selectAll()`</code> , <code>`clearSelection()`</code> , and selection state to edit actions.
<code>`DefaultDrawingView`</code>	Main Swing drawing view; implements <code>`EditableComponent`</code> and duplicates selected figures in normal drawing views.
<code>`AbstractDrawingView`</code>	Abstract drawing-view implementation with the same duplicate algorithm for subclasses using this view base.
<code>`Drawing`</code>	Container and mediator for figures; sorts selected figures, adds/removes figures, and emits undoable edit events.
<code>`Figure`</code>	Domain object cloned and transformed by the duplicate operation.
<code>`AbstractFigure`</code> and figure subclasses	Concrete figure behavior for cloning, transformation, bounds, attributes, and relationship remapping.
<code>`DefaultApplicationModel`</code>	Registers <code>`DuplicateAction.ID`</code> in the application action map.
<code>`DefaultMenuBuilder`</code>	Adds the Duplicate action to the Edit menu when the action exists.
SVG/ODG/PERT/NET panels and toolbars	Sample UI entry points that add Duplicate buttons/actions to user-facing sample applications.

3. Impact Analysis

The initial impact set was the code directly involved in starting and executing the Duplicate feature:

- `DuplicateAction`
- `AbstractSelectionAction`
- `EditableComponent`
- `DefaultDrawingView`
- `AbstractDrawingView`
- `Drawing`
- `Figure`

During static analysis, the feature was also found in application registration, menus, toolbar construction, and action labels. Those classes are part of the estimated impact set because a broken action id or changed action contract would affect the feature from the user interface.

Packages Visited

Package	Classes visited or counted	Contribution to feature	Change risk
`org.jhotdraw.action.edit`	13	Contains `DuplicateAction` and neighboring edit actions with duplicated active-component lookup.	Medium: user-facing commands share common behavior.
`org.jhotdraw.api.gui`	5	Defines `EditableComponent`, the action-to-view contract.	High: contract changes would ripple broadly. No contract change was made.
`org.jhotdraw.draw`	23	Contains drawing views and drawing model abstractions that perform actual duplication.	High: drawing view behavior affects many features. No change was made here.
`org.jhotdraw.draw.figure`	27	Figure domain objects are cloned, transformed, and remapped during duplication.	High: concrete figure changes can affect rendering and editing. No change was made here.
`org.jhotdraw.draw.io`	8	Input/output format strategies support clipboard and transfer behavior around drawing data.	Medium: related to copy/paste, not changed.
`org.jhotdraw.datatransfer`	10	Clipboard abstraction used by Copy; relevant because Copy shares the refactored lookup helper.	Low to medium: behavior covered by test, no production change here.
`org.jhotdraw.app`	14	Registers action ids and builds default application menus.	Low: no change needed because action id stayed stable.
`org.jhotdraw.samples.svg.gui`	33	SVG sample toolbar/menu entry points expose Duplicate in the lab GUI.	Low: no change needed because action construction stayed stable.

Scattering and Tangling

The Duplicate feature is scattered across action, API, drawing-view, drawing-domain, application, and sample UI packages. The action entry point is small, but the observable behavior depends on the drawing model and figure clone/remap behavior.

The most tangled areas are `org.jhotdraw.draw` and `org.jhotdraw.draw.figure`, because drawing views and figures support many other editing features. Changing the duplicate algorithm there would have a larger ripple effect. For that reason, the implementation deliberately avoided changing the domain algorithm and focused on action-layer duplication.

Estimated Impact Set

Before implementation, the expected production change set was:

Package	Expected changed classes	Reason
`org.jhotdraw.action.edit`	4	Pull duplicated active-component lookup into the common action base and update Duplicate, Copy, and Clear Selection.
`org.jhotdraw.actions` module POM	1	Add JUnit 4 for action-level tests.
`.github/workflows`	1	Add Maven CI workflow.
Root settings	1	Add Maven settings file for GitHub Packages credentials through environment variables.

4. Refactoring Patterns and Code Smells

Code Smell

The identified smell was duplicated code in the action layer. `DuplicateAction`, `CopyAction`, and `ClearSelectionAction` all repeated the same target-resolution pattern:

1. Use explicit target if present.
2. Otherwise ask the `KeyboardFocusManager` for the permanent focus owner.
3. Use it only if it is a `JComponent`.

This duplication is small but meaningful because edit actions are user-facing and more actions may need the same focus resolution. If the focus-resolution rule changes, duplicated copies can drift and create inconsistent action behavior.

Refactoring Pattern

The implemented refactoring is a combination of:

- Extract Method: isolate active component resolution.
- Pull Up Method: place that extracted method in `AbstractSelectionAction`.

The new shared method is:

```
protected JComponent getActiveComponent()
```

It returns the explicit target when present. Otherwise, it returns the permanent focus owner only when it is a `JComponent`.

Rationale

The refactoring keeps behavior unchanged while improving maintainability:

- `Duplicate`, `Copy`, and `Clear Selection` no longer duplicate focus lookup logic.
- `DuplicateAction.actionPerformed()` now reads as `Duplicate-specific logic` rather than plumbing.
- `Copy` can still act on disabled components because `CopyAction` still does not check `isEnabled()`.
- `Clear Selection` still supports both `EditableComponent` and `JTextComponent`.
- `Delete` was intentionally not changed because it extends `TextAction` and has a separate text deletion path.

5. Refactoring Implementation

Changed Production Classes

File	Change
<code>`AbstractSelectionAction.java`</code>	Added <code>`getActiveComponent()`</code> helper.
<code>`DuplicateAction.java`</code>	Replaced duplicated focus lookup with <code>`getActiveComponent()`</code> .
<code>`CopyAction.java`</code>	Replaced duplicated focus lookup with <code>`getActiveComponent()`</code> while preserving disabled-copy behavior.
<code>`ClearSelectionAction.java`</code>	Replaced duplicated focus lookup with <code>`getActiveComponent()`</code> .
<code>`jhotdraw-actions/pom.xml`</code>	Added JUnit 4.13.2 as a test dependency.

Added Support Files

File	Purpose
<code>`.maven-settings.xml`</code>	Allows Maven to authenticate to GitHub Packages using <code>`GITHUB_ACTOR`</code> and <code>`GITHUB_TOKEN`</code> environment variables.
<code>`.github/workflows/maven.yml`</code>	Runs Maven build/tests with Temurin JDK 11 for pushes and pull requests.
<code>`DuplicateActionTest.java`</code>	JUnit 4 tests for Duplicate, Clear Selection, and Copy behavior around the refactored helper.
<code>`report/gui-launch.log`</code>	Evidence from the SVG GUI smoke launch.
<code>`report/duplicate-maintenance-report.md`</code>	This report.

Estimate vs. Reality

The implementation matched the estimate closely.

Category	Estimated	Actual
Production Java classes changed	4	4
POM files changed	1	1
Test classes added	1	1
CI/settings files added	2	2
Drawing-domain classes changed	0	0

The most important actualization decision was to leave `DefaultDrawingView`, `AbstractDrawingView`, `Drawing`, and `Figure` untouched. That avoided unnecessary risk in the highly tangled drawing domain.

Clean Code, Clean Architecture, and SOLID Context

Clean Code:

- The action methods now contain less repeated plumbing and more feature-specific intent.
- The helper name `getActiveComponent()` describes the shared concept directly.
- Duplicate, Copy, and Clear Selection are easier to compare because their common setup code has been removed.

Clean Architecture:

- The action layer remains an outer UI/control layer.
- The domain behavior remains in drawing views and figures.
- No dependency direction was inverted or worsened; the action still depends on the `EditableComponent` abstraction rather than concrete drawing classes.

SOLID:

- Single Responsibility Principle: active component lookup is now the responsibility of the shared action abstraction, not each concrete action.
- Open/Closed Principle: future selection actions can reuse the helper without rewriting lookup logic.
- Liskov Substitution Principle: concrete actions remain valid subclasses of `AbstractSelectionAction` and keep their existing action behavior.
- Interface Segregation Principle: `EditableComponent` was not expanded; tests use only the existing contract.
- Dependency Inversion Principle: `DuplicateAction` still depends on `EditableComponent`, not on `DefaultDrawingView`.

6. Verification

Automated Tests Added

Test class:

```
jhotdraw-actions/src/test/java/org/jhotdraw/action/edit/DuplicateActionTest.java
```

Test cases:

Test	Verifies
<code>`duplicateDelegatesToEnabledEditableTarget`</code>	Duplicate calls <code>`duplicate()`</code> exactly through the editable target and not unrelated edit methods.
<code>`duplicateIgnoresDisabledEditableTarget`</code>	Duplicate does not act on disabled editable targets.
<code>`clearSelectionDelegatesToEnabledEditableTarget`</code>	Clear Selection still delegates to <code>`clearSelection()`</code> .
<code>`clearSelectionCollapsesTextSelection`</code>	Clear Selection still supports <code>`JTextComponent`</code> .
<code>`copyStillExportsDisabledTargetToClipboard`</code>	Copy still exports from disabled targets after helper extraction.

Commands and Results

Portable tool verification:

```
Apache Maven 3.8.8
Java version: 11.0.31, vendor: Eclipse Adoptium
```

Module test:

```
mvn -s .maven-settings.xml -pl jhotdraw-actions test
```

Result:

```
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Full reactor test:

```
mvn -s .maven-settings.xml test
```

Result:

```
BUILD SUCCESS
```

Full clean install:

```
mvn -s .maven-settings.xml clean install
```

Result:

```
BUILD SUCCESS
```

Test coverage is focused on the refactoring surface. It does not deeply retest the figure-cloning algorithm because that algorithm was intentionally not changed.

7. BDD Scenarios

JGiven was not added because the useful automation surface for this refactoring is small and already covered by JUnit 4. The BDD scenarios are still mapped formally from the user story and can be automated later with JGiven stages if required.

Scenario 1: Duplicate selected drawing elements

Given I have selected one or more figures in an enabled drawing view

When I trigger the Duplicate action

Then the drawing view duplicates the selected figures

And the duplicated figures are offset from the originals

And the duplicated figures become the selected figures

And an undoable edit is registered for the operation

Scenario 2: Ignore disabled target

Given the Duplicate action has a disabled editable target

When I trigger the Duplicate action

Then the target's `duplicate()` operation is not called

And no drawing state is changed

Scenario 3: Preserve neighboring edit action behavior

Given Copy and Clear Selection share the same active-component lookup mechanism

When Copy is triggered on a disabled target

Then Copy still exports through the target transfer handler

When Clear Selection is triggered on a text component

Then the selected text range is collapsed

8. Conclusion

The Duplicate feature was selected, located, analyzed, refactored, and verified according to the software maintenance process from the course:

Initiation -> Concept Location -> Impact Analysis -> Prefactoring -> Actualization -> Postfactoring -> Conclusion.

The implemented maintenance change improves the action layer by removing duplicated focus-resolution logic from three edit actions and centralizing it in `AbstractSelectionAction`. The observable behavior of Duplicate, Copy, and Clear Selection is preserved by focused JUnit tests and full Maven verification.

The final change is intentionally low risk:

- No public action ids changed.
- No `EditableComponent` contract changed.
- No drawing-domain duplication algorithm changed.
- No sample GUI registration changed.
- CI was added so future pull requests build and test with JDK 11.

Remaining risk:

- GitHub Projects could not be used directly because the current GitHub CLI token was missing project scopes. A GitHub issue was created as a backlog evidence item instead.
- CI run evidence is available from the successful push and pull-request workflow runs linked at the top of this report.
- JGiven automation was not added; BDD scenarios are documented and the refactoring behavior is covered by JUnit 4.

Future maintenance recommendation:

Review other edit actions such as Select All, Cut, Paste, and Delete for similar active-component lookup duplication. Delete should be handled carefully because it extends `TextAction` and includes text-specific deletion behavior.

Appendix A. Detailed Course Process Alignment

The course material describes software change as an iterative maintenance process rather than a single code edit. The important phases are initiation, concept location, impact analysis, prefactoring, actualization, postfactoring, verification, and conclusion. This report follows that flow directly.

Initiation is represented by the Duplicate user story and the backlog issue. The user story is intentionally short and stakeholder-oriented. It does not mention `DuplicateAction`, `EditableComponent`, or `DefaultDrawingView`; it only states the user's goal: duplicate selected drawing elements to reuse existing shapes. That separation matters because the maintenance process starts from a change request, not from an implementation idea.

Concept location is represented by the static search and source-code trace from the action id to the drawing-view implementation. The course emphasizes that features are often started from controller classes and then lead into domain classes. Duplicate fits that pattern. The controller/action class is `DuplicateAction`; the contract boundary is `EditableComponent`; the domain behavior is in drawing views and figures. This is why the report separates action-layer classes from domain-layer classes.

Impact analysis is represented by the package table and the decision to avoid changing the drawing-domain algorithm. The concept location found that the feature depends on `Drawing`, `Figure`, and drawing-view state. Those packages are more tangled because they support many editing features, not just Duplicate. A maintenance change that touches them would need broader regression testing. The safer impact boundary for this assignment was therefore the action layer, where the code smell existed and where the duplicated logic could be removed without changing figure duplication behavior.

Prefactoring is represented by the code smell identification before the implementation. The relevant smell was duplicated active-component lookup. It appeared in several edit actions and made the code harder to keep consistent. The report does not claim that the Duplicate feature was broken. The purpose of the work was maintainability: improve the internal structure without changing observable behavior.

Actualization is represented by the implementation of `getActiveComponent()` and by the updates to `DuplicateAction`, `CopyAction`, and `ClearSelectionAction`. The code change is small because the

impact analysis deliberately contained it. The new helper is placed in the abstraction that already owns the `target` field and selection-action behavior.

Postfactoring is represented by checking that the changed actions are still readable and by not adding unnecessary abstractions. The helper is not a new service object or framework. It is a protected method in the existing superclass, matching the local style of the codebase. This keeps the refactoring proportionate to the problem.

Verification is represented by unit tests, full Maven builds, CI runs, and the GUI smoke launch. The course material stresses that verification spans multiple phases and that a successful compile is not enough. For this reason, the work includes both local Maven verification and GitHub Actions evidence.

Conclusion is represented by the final pull request, successful CI, and this report. The conclusion phase creates a new baseline candidate. In this case the candidate baseline is the branch `maintenance/duplicate-feature`, pushed to the fork and opened as a draft pull request.

Appendix B. Expanded Concept Location Notes

Concept location was performed with a combination of static search and runtime-oriented reasoning. The starting point was the user-visible term "Duplicate", because the feature is visible in menus and toolbars. Searching for `edit.duplicate` and `duplicate()` found the action id, labels, menu registration, toolbar usage, interface contract, and drawing-view implementations.

Static Search Results Interpreted

The search found `edit.duplicate` in label resource bundles. These entries define the visible menu/action text but do not implement the behavior. They are still relevant because a user-facing feature depends on action ids and labels remaining stable. If the action id changed without updating menus and labels, the feature could disappear from the UI even if the domain method still existed.

The search found `DuplicateAction` in the application model and menu builder. This showed that Duplicate is not only a sample toolbar button; it is part of the default edit action set. `DefaultApplicationModel` registers the action under `DuplicateAction.ID`, and `DefaultMenuBuilder` adds it to the Edit menu if the action exists. That makes `DuplicateAction.ID` part of the public behavior of the application layer.

The search found `DuplicateAction` in several sample panels and toolbars. This matters for the lab because the SVG sample application is used to launch the GUI. The same action class is reused by the sample UI, so action-layer changes affect the lab's visible application.

The search found `EditableComponent.duplicate()`. This is the key contract boundary. `DuplicateAction` does not know how to duplicate figures. It only knows that an editable component can duplicate its selected region. This is a clean separation: the action can be generic, and the drawing view can own drawing-specific duplication.

The search found `DefaultDrawingView.duplicate()` and `AbstractDrawingView.duplicate()`. These methods contain the actual algorithm. They sort the selected figures, clone each figure, translate clones by five pixels in both directions, remap relationships, select duplicates, and register undo/redo behavior. This is the domain core of the Duplicate feature.

Feature Call Chain

Step	Code location	Responsibility
1	Menu, toolbar, or key binding	User triggers Duplicate from the UI.
2	<code>`DuplicateAction.actionPerformed()`</code>	Finds active target and checks that it is enabled and editable.
3	<code>`EditableComponent.duplicate()`</code>	Contract method called by the action.

Step	Code location	Responsibility
4	<code>`DefaultDrawingView.duplicate()`</code> or <code>`AbstractDrawingView.duplicate()`</code>	Performs drawing-specific duplication.
5	<code>`Drawing.sort(getSelectedFigures())`</code>	Orders selected figures according to drawing order.
6	<code>`Figure.clone()`</code>	Creates a copy of each selected figure.
7	<code>`Figure.transform(AffineTransform)`</code>	Offsets the clone so it is visible beside the original.
8	<code>`Drawing.add(Figure)`</code>	Adds each clone to the model.
9	<code>`Figure.remap(...)`</code>	Reconnects copied relationships between cloned figures when needed.
10	<code>`addToSelection(duplicates)`</code>	Makes duplicates the active selection.
11	<code>`fireUndoableEditHappened(...)`</code>	Registers undo/redo support for the duplicate operation.

Dynamic Analysis Plan

The lab asks for debugger-based concept location. The following debugger procedure documents how this feature should be inspected in an IDE such as IntelliJ:

1. Set a breakpoint in `DuplicateAction.actionPerformed()`.
2. Launch `org.jhotdraw.samples.svg.Main` from `jhotdraw-samples-misc`.
3. Select or create a figure in the SVG drawing view.
4. Trigger Duplicate from the toolbar or Edit menu.
5. Confirm that the breakpoint in `DuplicateAction` is hit.
6. Step into `EditableComponent.duplicate()`.
7. Confirm that the concrete receiver is a drawing view implementation.
8. Step into `DefaultDrawingView.duplicate()` or `AbstractDrawingView.duplicate()`.
9. Watch `getSelectedFigures()` to confirm the selected figure set.
10. Watch `duplicates` to confirm cloned figures are added.
11. Step through `remap()` when duplicating connected figures.
12. Confirm that the undoable edit is fired.

The GUI smoke run in this work confirms that the sample application can launch. A full manual debugger recording was not committed as code, but the static trace above identifies exactly where the runtime breakpoints should be placed.

Domain Concepts Found

The Duplicate feature is built from several domain concepts:

Selection: The user chooses one or more figures. This is represented by `getSelectedFigures()` in the drawing view.

Figure identity: The original figures and cloned figures are distinct objects. The original-to-duplicate map preserves that relationship during remapping.

Drawing order: The sorted selection ensures that duplicated figures respect the drawing's ordering rules.

Geometric offset: The duplicate operation translates clones by (5, 5) so the result is visible and not perfectly hidden behind the original.

Relationship remapping: Connected or composite figures may refer to other figures. Remapping updates references from originals to clones where possible.

Undoable edit: The operation is not just a model mutation; it is a user command that must support undo and redo.

Appendix C. Expanded Impact Analysis

Impact analysis estimates what may need to change and what may be indirectly affected. It is especially important in JHotDraw because the framework is deliberately reusable: action classes, drawing views, figures, tools, and sample applications are connected through stable interfaces.

Initial Impact Set

The initial impact set was created from direct concept-location results:

Class or interface	Why it was included
<code>`DuplicateAction`</code>	Direct feature entry point.
<code>`AbstractSelectionAction`</code>	Superclass containing target state and selection-action behavior.
<code>`EditableComponent`</code>	Contract that exposes <code>`duplicate()`</code> to actions.
<code>`DefaultDrawingView`</code>	Concrete drawing view implementation with duplicate algorithm.
<code>`AbstractDrawingView`</code>	Abstract drawing view implementation with a similar duplicate algorithm.
<code>`Drawing`</code>	Model container used for sorting, adding/removing figures, and undoable edit mediation.
<code>`Figure`</code>	Cloned, transformed, and remapped by the feature.

Extended Impact Set

The extended impact set includes classes that may be affected by changes to the initial set:

Area	Reason for inclusion
Application action registration	The feature is registered by action id. If action id changes, UI lookup breaks.
Menu builder	The Edit menu includes Duplicate when the action exists.
Sample toolbars and panels	Lab-visible applications construct Duplicate actions directly.
Label resource bundles	User-visible text depends on <code>`edit.duplicate`</code> keys.
Transfer/clipboard code	Not directly part of Duplicate, but Copy shares the refactored active-target helper.
Text components	Clear Selection supports <code>`JTextComponent`</code> , so the refactoring must not narrow behavior to drawing views only.

Risk by Layer

Layer	Risk if changed	Reason
Action layer	Low to medium	User-facing behavior is visible, but the code is small and testable with fake components.
API contract layer	High	Any change to <code>`EditableComponent`</code> affects all editable components and actions.
Drawing view layer	High	Selection, undo, repainting, and figure management are shared across many features.

Layer	Risk if changed	Reason
Figure layer	High	Cloning, transformation, and remapping are core behaviors used by many features.
UI registration layer	Medium	A wrong action id can hide or disconnect a feature even if code compiles.
CI/settings layer	Low	Build configuration changes are visible but easy to verify through CI.

Why the Drawing Algorithm Was Not Changed

The duplicated active-component lookup was in the action layer, while the actual figure duplication algorithm is in the drawing layer. It would have been tempting to refactor the duplicate algorithm as well, because `DefaultDrawingView` and `AbstractDrawingView` contain similar logic. That was intentionally not done in this iteration.

The reason is impact containment. Changing the drawing algorithm would require deeper tests with actual figure implementations, selection state, undo managers, connection figures, and repaint behavior. It could also affect subclasses that rely on the current algorithm's exact side effects. The course process does not reward large changes for their own sake. A maintainable change should be scoped to the identified smell and verified appropriately.

This is why the final implementation changed only the active-component lookup path and left the feature's business behavior intact.

Ripple Effect Discussion

If `getActiveComponent()` were implemented incorrectly, the ripple effect would primarily affect actions that use it. In this work, that set is only Duplicate, Copy, and Clear Selection. The tests were chosen to cover the different behavioral rules among these actions:

- Duplicate requires an enabled editable target.
- Copy permits disabled targets.
- Clear Selection handles both editable components and text components.

This test selection is important because the helper is shared but the actions are not identical. A naive refactoring could accidentally impose Duplicate's enabled/editable condition on Copy, or remove Clear Selection's text behavior. The unit tests prevent those regressions.

If `EditableComponent` had been changed, the ripple effect would have been larger. Every editable component and every edit action could be affected. Because the existing interface was sufficient, the contract remained unchanged.

If `DefaultDrawingView.duplicate()` had been changed, the ripple effect could include drawing order, undo/redo behavior, connection remapping, and selection state. That would require integration-level drawing tests. Since that code was not changed, the report treats it as concept-location evidence rather than implementation scope.

Appendix D. Code Smell and Refactoring Evidence

The repeated active-component lookup looked like this conceptually:

```

JComponent c = target;
if (c == null && KeyboardFocusManager.getCurrentKeyboardFocusManager()
    .getPermanentFocusOwner() instanceof JComponent) {
    c = (JComponent) KeyboardFocusManager.getCurrentKeyboardFocusManager()

```

```

        .getPermanentFocusOwner();
    }

```

This appeared in multiple action classes. The problem is not only that the same lines were repeated. The deeper problem is that a rule about how edit actions find their target was scattered across concrete actions. Scattered rules are harder to audit and harder to change safely.

The refactoring introduced:

```

protected JComponent getActiveComponent()

```

The concrete actions now express only their action-specific behavior:

```

JComponent c = getActiveComponent();
if (c != null && c.isEnabled()) {
    if (c instanceof EditableComponent) {
        ((EditableComponent) c).duplicate();
    } else {
        c.getToolkit().beep();
    }
}

```

Before and After Comparison

Class	Before	After
<code>`DuplicateAction`</code>	Mixed focus lookup with duplicate behavior.	Uses shared lookup, then performs duplicate-specific checks.
<code>`CopyAction`</code>	Mixed focus lookup with clipboard export behavior.	Uses shared lookup, still allows disabled targets.
<code>`ClearSelectionAction`</code>	Mixed focus lookup with editable/text clearing behavior.	Uses shared lookup, still supports editable and text targets.
<code>`AbstractSelectionAction`</code>	Owned <code>`target`</code> but did not expose target-resolution behavior.	Owns both <code>`target`</code> and active-component resolution.

Maintainability Improvement

The maintainability gain is local but real:

- There is one implementation of the focus fallback rule.
- Concrete actions are shorter.
- Future action tests can reuse the same expectations for active target lookup.
- Reviewing action behavior is easier because duplicated setup code no longer obscures the command-specific branch.

Why This Is Behavior-Preserving

The helper returns the same component that each concrete action previously computed. The action-specific conditions remain in the concrete actions. That distinction is what preserves behavior.

For Duplicate, the action still checks `c != null && c.isEnabled()` and then checks `c instanceof EditableComponent`.

For Copy, the action still does not check `isEnabled()`. This preserves the original comment that copying is allowed for disabled components.

For Clear Selection, the action still checks `isEnabled()` and still handles both `EditableComponent` and `JTextComponent`.

Appendix E. Verification Details and Evidence

Verification was done at several levels because the course material emphasizes that compiling is not the same as proving behavior.

Build Verification

The baseline skipped-test build verified that the repository and dependency setup worked before the refactoring:

```
mvn -s .maven-settings.xml clean install -DskipTests
```

The post-change clean install verified that the whole reactor builds, tests, packages, and installs:

```
mvn -s .maven-settings.xml clean install
```

Both commands completed successfully.

Unit Verification

The new JUnit 4 tests are action-level tests. They do not require a full Swing drawing window. This is intentional because the changed code is action-layer target resolution, not drawing-domain cloning.

The fake editable component counts method calls. This directly verifies that `Duplicate` delegates to `duplicate()` and does not accidentally call `delete()`, `selectAll()`, or `clearSelection()`.

The disabled target test verifies the boundary case where the target exists but should not be acted on. This is a relevant boundary because the original `Duplicate` behavior explicitly checked `isEnabled()`.

The Copy test uses `ClipboardUtil.setClipboard(new Clipboard("test clipboard"))` so it does not depend on the operating-system clipboard. That keeps the unit test stable in CI and avoids GUI-environment flakiness.

The Clear Selection text test verifies that the refactoring did not narrow Clear Selection to only `EditableComponent`. This is important because the edit actions are documented as acting on editable components and text components.

Regression Verification

The full reactor test command ran existing tests in other modules as well as the new action tests:

```
mvn -s .maven-settings.xml test
```

The existing TestNG tests in `jhotdraw-utils` and `jhotdraw-core` passed, and the new JUnit 4 tests in `jhotdraw-actions` passed. This gives regression evidence that the refactoring did not break the existing test suite.

Continuous Integration Verification

The GitHub Actions workflow runs on push and pull request events. It uses Temurin JDK 11 and Maven cache. The workflow command is:

```
mvn -s .maven-settings.xml clean install
```

The final commit triggered two successful workflow runs:

- Push run: <https://github.com/Tokushiro/JHotDraw/actions/runs/27103509067>
- Pull-request run: <https://github.com/Tokushiro/JHotDraw/actions/runs/27103509953>

The workflow uses `GITHUB_TOKEN` and `.maven-settings.xml` so Maven can resolve packages from the GitHub Packages repository configured by the course project.

GUI Smoke Verification

The SVG sample was launched with:

```
mvn -s ../..maven-settings.xml exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"
```

The process remained alive after 12 seconds and was then stopped intentionally. A short-lived crash would have ended the process. The log contains resource warnings for missing optional icons but no startup exception. That is acceptable smoke evidence for the lab requirement that the GUI starts.

Verification Limits

The tests do not prove that every concrete figure clone/remap combination works. That was outside the implementation scope because the figure-domain algorithm was not changed. A deeper future test suite could add integration tests using actual drawing views and figures to verify:

- Duplicating one rectangle creates exactly one new rectangle.
- Duplicating multiple selected figures preserves drawing order.
- Duplicating connected figures remaps connections between cloned figures.
- Undo removes duplicated figures.
- Redo re-adds duplicated figures.

Those tests would be valuable if a future change touches `DefaultDrawingView.duplicate()` or `AbstractDrawingView.duplicate()`.

Appendix F. Exam-Oriented Review Notes

This section prepares the report for oral or written exam discussion. The course transcript notes that exam review questions may ask for definitions, examples, and reasoning about the portfolio work. The following answers are tied to this maintenance task.

What is refactoring?

Refactoring is a disciplined change to the internal structure of code without changing its observable external behavior. In this project, the observable behavior is that Duplicate, Copy, and Clear Selection still work as before. The internal structure changed because active-component lookup moved from concrete actions into the superclass.

What is a code smell?

A code smell is a sign that code may be harder to understand, modify, or verify than necessary. It is not always a bug. Here, the smell was duplicated code. Several actions repeated the same focus-owner lookup logic. The duplication increased maintenance cost because each copy would need to be checked if target-resolution behavior changed.

What refactoring pattern was used?

The work used Extract Method and Pull Up Method. The repeated lookup logic was extracted into a method, then placed in the common superclass `AbstractSelectionAction`. This matches the inheritance structure already present in the codebase.

Why is impact analysis important?

Impact analysis estimates the consequences of a change before implementation. Without it, this task could have expanded into the drawing domain and risked breaking figure cloning, undo, or selection behavior. The analysis showed that the maintainability smell could be addressed in the action package while leaving the high-risk drawing packages unchanged.

What is concept location?

Concept location is finding where a domain concept or feature is implemented in code. For Duplicate, the feature concept starts in the action layer but the drawing behavior is implemented in drawing views and figures. The concept location result is therefore a chain, not a single class.

What is scattering?

Scattering means a feature is implemented across multiple packages or classes. Duplicate is scattered across action registration, action behavior, editable component contracts, drawing views, figures, labels, and sample UI panels.

What is tangling?

Tangling means one package or class supports multiple features. `org.jhotdraw.draw` is tangled because drawing views, figures, selection, undo, and model operations support many features besides Duplicate. That is why it was treated as a high-risk package.

How did CI help?

CI provided automated build and test verification on GitHub for the pushed branch and draft pull request. This reduces the "works on my machine" risk and creates evidence that the branch can build in a clean GitHub-hosted environment.

How does this relate to SOLID?

The strongest SOLID relation is SRP and OCP. `AbstractSelectionAction` now owns the shared target-resolution responsibility. Concrete actions remain focused on their command. Future selection actions can reuse the helper without copying code.

How does this relate to Clean Architecture?

The action layer remains separate from the drawing domain. `DuplicateAction` does not depend on `DefaultDrawingView`; it depends on the `EditableComponent` abstraction. The drawing-specific behavior remains inside drawing views and figures.

What would be the next maintenance improvement?

The next improvement would be to inspect other edit actions for the same lookup duplication. `SelectAllAction`, `CutAction`, and `PasteAction` may have related patterns. `DeleteAction` should be handled separately because it extends `TextAction` and contains text-deletion logic.

Appendix G. Evidence Checklist

Requirement	Evidence
Fork repository	<code>`https://github.com/Tokushiro/JHotDraw`</code>
Branch	<code>`maintenance/duplicate-feature`</code>
User story/backlog item	<code>`https://github.com/Tokushiro/JHotDraw/issues/1`</code>
Pull request	<code>`https://github.com/Tokushiro/JHotDraw/pull/2`</code>
JDK 11 used	Temurin 11.0.31 output from local Maven verification
Maven 3.8.x used	Apache Maven 3.8.8 output from local Maven verification
Baseline build	<code>`mvn -s .maven-settings.xml clean install -DskipTests`</code> , successful
Unit tests	<code>`DuplicateActionTest`</code> , 5 tests passing
Full local verification	<code>`mvn -s .maven-settings.xml clean install`</code> , successful
CI verification	Final push and PR workflow runs successful
GUI launch	<code>`report/gui-launch.log`</code> and <code>`report/gui-launch.err.log`</code>
Markdown report	<code>`report/duplicate-maintenance-report.md`</code>
PDF report	<code>`report/duplicate-maintenance-report.pdf`</code>

Appendix H. Future Work and Risks

The current change is intentionally small, but the concept location revealed future maintenance opportunities.

First, the duplicate algorithm appears in both `DefaultDrawingView` and `AbstractDrawingView`. This may be a deeper duplication smell. It was not changed because it touches the drawing domain, but a future iteration could investigate whether a shared drawing-view helper or template method is appropriate.

Second, the action package contains several edit actions with similar focus-target behavior. This iteration changed only the three actions needed to remove the confirmed duplication while preserving behavior with tests. A future iteration could inspect `Select All`, `Cut`, `Paste`, and `Delete`. `Delete` should be approached carefully because of its `TextAction` inheritance and text-specific behavior.

Third, the BDD scenarios are documented but not automated with `JGiven`. Automating them would be most valuable if the assignment requires executable acceptance documentation. The likely `JGiven` stage structure would be:

- `GivenDrawingWithSelectedFigures`
- `WhenDuplicateIsTriggered`
- `ThenDuplicatedFiguresAreSelected`

Those stages would need either actual drawing views and figure instances or carefully designed fakes. Because the current production change is in the action layer, the `JUnit` tests provide stronger direct evidence for this iteration.

Fourth, the GitHub Project card could not be created because the GitHub token lacked project scopes. The issue fallback is acceptable as backlog evidence, but if strict GitHub Projects evidence is required, the account should run:

```
gh auth refresh -s project
```

After refreshing scopes, the existing issue can be added to a GitHub Project board without changing the code.

Fifth, report evidence should be kept with the repository. This branch includes both Markdown and PDF. The Markdown is easier to maintain; the PDF is the submission artifact.